Modular Separator Weighted Model Counting for Neuro-Symbolic Inference over Large Ontologies

Tengwei Song
ELLIS Institute Finland & Åbo Akademi University
Tengwei.Song@abo.fi

Submitted to *Journal of Artificial Intelligence Research*, October 2026

Abstract

Neural networks that assign scores to concepts governed by a symbolic ontology face a consistency problem at inference time: a protein may receive a higher predicted probability of belonging to a specialized cellular-component term than to the broader term containing it, even though the ontology mandates that any member of the narrower class must also be a member of the broader one. *Weighted model counting* (WMC) addresses this by treating predictor scores as Bernoulli parameters, summing the weights of all assignments that satisfy the ontology, and renormalizing. The result is a principled posterior distribution whose support is confined to logically valid labelings, computed entirely from fixed predictor scores without any retraining.

The computational bottleneck is structural width: exact WMC is exponential in the treewidth of the constraint graph, and large biomedical ontologies contain thousands of reconvergent regions where independent messages from different paths double-count shared evidence, making exact inference intractable for monolithic solvers and path-wise approximations inaccurate.

We introduce *modular separator WMC*, a framework that decomposes the constraint graph into a small core and peripheral modules, eliminates each module into a joint boundary factor (separator), and contracts a reduced core. Exact separators provably recover the global WMC posterior. For wide boundaries, we construct nonnegative tensor-train (TT) separators by querying a boundary-conditioned oracle directly, project logical support separately, and inject the factorized TT into reduced-core inference without materializing dense tables. Recursive core contraction handles ontologies whose reduced core remains intractable at the first level.

We prove that exact separators preserve the monolithic WMC posterior (Proposition 6.1) and that module marginal error is bounded by boundary posterior error under TV distance (Proposition 6.2). Experiments on five complete biomedical ontologies—

Gene Ontology cellular component (GO cc), molecular function (GO mf), and biological process (GO bp); UBERON anatomy; and ChEBI chemistry, comprising up to 205,592 terms—show exact marginal recovery on instances where the decomposition is non-trivial (recursion depth > 0), completion of all-term inference beyond the 300 GB memory limit of monolithic junction-tree inference, and systematic reduction in recovered-marginal error as separator rank increases. On the CAFA Critical Assessment of Function Annotation benchmark applied to GO cc, ontology-conditioned posterior correction eliminates true-path violations and improves calibration over raw predictor scores and loopy belief propagation.

1. Introduction

Neuro-symbolic prediction tasks pair a neural scorer with a symbolic knowledge base that restricts which output configurations are valid. In *hierarchical multi-label classification*, a classifier assigns independent scores to labels arranged in a taxonomy, but the taxonomy itself demands that any positive prediction at a specific concept implies positive predictions at all of its ancestors [Silla and Freitas, 2011]. In protein-function prediction, the Gene Ontology (GO) encodes is-a relations among hundreds of thousands of molecular-function, biological-process, and cellular-component terms: a protein predicted to perform a highly specialized enzymatic activity must also be predicted to perform the general biochemical processes containing it [Ashburner et al., 2000, The Gene Ontology Consortium, 2023]. More broadly, ontology-based classification—across anatomy [Mungall et al., 2012], chemistry [Hastings et al., 2016], phenotype, and disease—faces the same structural issue: a vector of local predictor scores is inconsistent with probability one, because independent scoring ignores the implication structure of the ontology. We call any complete assignment satisfying all ontology constraints a *valid labeling*; raw predictor scores typically assign nonzero probability mass to millions of invalid labelings.

Weighted model counting (WMC) provides a principled, training-free mechanism to correct this inconsistency. Given a Boolean formula T (encoding the ontology constraints) and nonnegative literal weights (derived from predictor scores), WMC sums the weights of all satisfying assignments of T [Chavira and Darwiche, 2008]. Interpreting the predictor output $p_i \in (0, 1)$ for ontology term i as the Bernoulli parameter of $y_i \in \{0, 1\}$, the product reference distribution $q_p(y) = \prod_i p_i^{y_i}(1 - p_i)^{1-y_i}$ has support over all 2^m assignments. Normalizing the restriction of q_p to valid labelings defines the *WMC posterior*

$$p_T(y) = \frac{\mathbf{1}[y \models T]\, q_p(y)}{Z_T(p)}, \qquad Z_T(p) = \mathrm{WMC}(T, p) = \sum_{y \models T} q_p(y).$$

Every marginal $\mu_i = p_T(y_i = 1)$ and every Boolean query uses the same normalizer $Z_T(p)$, so the corrected scores are globally consistent by construction. This *inference-time* correction requires no modification to the predictor architecture, training procedure, or loss function: it is a plug-in posterior layer applied to fixed scores at test time.

Despite its conceptual appeal, exact WMC is #P-hard in general [Valiant, 1979], and practical solvers based on knowledge compilation or variable elimination can require time and memory exponential in the *treewidth* of the constraint graph G_T [Bodlaender, 1993, Darwiche and Marquis, 2002, Darwiche, 2011]. State-of-the-art WMC solvers such as ADDMC, DPMC, and d4 [Dudek et al., 2020a,b, Lagniez and Marquis, 2017] achieve remarkable scalability on random or industrial instances, but biomedical ontologies expose a different structural regime: the number of variables is orders of magnitude larger than in benchmark suites, while the logical structure—is-a implications forming a directed acyclic graph —is comparatively simple. GO biological process encodes 25,950 terms with thousands of reconvergent paths where multiple specialized terms share a common ancestor through different routes. On such inputs, a 300 GB junction-tree backend exhausts its memory budget before completing even a single ontology.

The cheaper alternative—loopy belief propagation (BP) [Pearl, 1988]—passes messages along individual paths and merges them independently at each variable. This is computationally efficient but formally incorrect on reconvergent structures. When two paths from a common ancestor d arrive at a shared descendant a through intermediates b and c, messages along $d \to b \to a$ and $d \to c \to a$ both carry evidence from d; treating them as independent double-counts that evidence and changes the posterior defined by global WMC. We make this failure mode precise in Section 4 with a four-variable example that shows the path-wise estimate and the exact WMC marginal diverging by more than 8% relative error.

Compositional WMC methods [Dal et al., 2017, 2021] reduce cost by partitioning a formula into components that communicate through separator variables. These methods do not address the regime where boundary sizes routinely reach hundreds or thousands of variables, as observed in GO biological process (max $|B_j| = 624$) and ChEBI ($|B_j| = 1{,}082$) in our experiments. Dense separator tables are infeasible at these widths. Low-rank tensor representations have been applied to graphical model inference [Novikov et al., 2014, Wrigley et al., 2017] and knowledge compilation [Goral et al., 2024, Onaka et al., 2025], but direct construction of a TT separator from boundary-conditioned oracle queries—without compiling the full dense table first—is not covered by existing frameworks.

We introduce *modular separator WMC* to address both requirements. Our framework proceeds in three stages. **Boundary-aware decomposition**: a core-selection algorithm identifies variables C such that the remaining variables partition into modules—connected components of $G_T \setminus C$—each with a boundary-constrained width within a prescribed cap. **Module elimination and separator construction**: each module M_j is eliminated by summing out its interior while holding boundary variables B_j fixed, producing a separator factor $F_j(y_{B_j})$—a joint function capturing all dependence the module induces on the rest of the ontology. For wide boundaries, we build a nonnegative tensor-train separator by querying the factor at sampled boundary assignments and fitting a rank-r TT using alternating nonnegative least squares; logical support is maintained separately. **Core contraction and marginal recovery**: the original normalizer is recovered by contracting the reduced core with one separator factor per eliminated module. A calibrated module junction tree then reuses its messages to recover every eliminated variable's marginal through backward

substitution.

Contributions.

1. We introduce modular separator WMC, which decomposes the constraint graph into a small core and peripheral modules, eliminates each module into a joint separator factor, and recovers every ontology-term marginal through backward calibration. The framework handles exact and approximate separators through a common interface.

2. We construct wide-boundary separators directly from boundary-conditioned module-WMC queries using a nonnegative tensor-train fit, projecting logical support separately to preserve constraint correctness independent of approximation rank.

3. We prove exact modular contraction (Proposition 6.1) and stability of module recovery under approximate boundary posteriors (Proposition 6.2).

4. We validate the framework on five complete biomedical ontologies. On instances where the decomposition is non-trivial (recursion depth > 0, boundary size > 0), exact separators recover all reference marginals to machine precision. On instances where monolithic inference is infeasible, rank-controlled TT separators complete all-term inference within practical budgets. On the CAFA benchmark, posterior correction eliminates constraint violations and improves calibration.

Paper organization. Section 2 reviews related work. Section 3 establishes notation and the core-module-boundary decomposition. Section 4 motivates joint separators with a reconvergence example. Section 5 presents the full framework. Section 6 states and proves the exactness and stability guarantees. Section 7 reports experiments. Section 8 discusses the role of ontology structure, limitations, and future directions. Section 9 concludes.

2. Related Work

2.1. Constrained and Ontology-Based Prediction

Hierarchical multi-label classification (HMC) requires that positive predictions respect a taxonomic parent-child ordering [Silla and Freitas, 2011]. Methods for enforcing this constraint range from architecture design [Giunchiglia and Lukasiewicz, 2020] to constraint-aware losses such as semantic loss [Xu et al., 2018] and SPL [Ahmed et al., 2022], to posterior regularization [Ganchev et al., 2010] and constrained decoding [Diligenti et al., 2017]. Correction methods applied at inference time include upward max-closure [Obozinski et al., 2008] and sum-product belief propagation over the constraint graph [Giunchiglia and Lukasiewicz, 2020].

In protein-function prediction, methods such as DeepGO [Kulmanov et al., 2018], DeepGOZero [Kulmanov and Hoehndorf, 2022], and DeepGOCNN [Kulmanov and Hoehndorf, 2020] assign per-term scores evaluated under the CAFA framework [Radivojac et al., 2013,

Zhou et al., 2019]. Post-processing methods—including max-closure and loopy BP—are applied to improve constraint satisfaction after training. Our work studies WMC as a plug-in inference layer in this setting, targeting the same fixed-score correction objective while returning a fully normalized posterior rather than a constraint-satisfying point estimate.

2.2. Exact Weighted Model Counting

Weighted model counting computes the sum of weights of satisfying assignments of a Boolean formula [Chavira and Darwiche, 2008]. It generalizes model counting (#SAT) to real-valued weights and underlies probabilistic inference in graphical models, Bayesian networks, and probabilistic logic programs [Fierens et al., 2015, De Raedt et al., 2007].

Knowledge compilation translates a formula into a tractable circuit representation that supports WMC in linear time in the circuit size. Representative target languages include binary decision diagrams (BDDs), deterministic decomposable negation normal form (d-DNNF) [Darwiche and Marquis, 2002], and sentential decision diagrams (SDDs) [Darwiche, 2011]. Variable elimination approaches such as ADDMC [Dudek et al., 2020a], DPMC [Dudek et al., 2020b], and sharpSAT-TD process the formula directly, avoiding a separate compilation step. The d4 compiler [Lagniez and Marquis, 2017] interleaves clause learning with compilation and performs competitively on industrial instances. These solvers are evaluated annually at the Model Counting Competition [Fichte et al., 2021]; we include ADDMC, DPMC, and d4 in our experimental comparison (Section 7.4).

All of these methods treat the full variable set monolithically: they do not exploit ontology-specific structure (sparse DAG, large scale, predictor-valued weights) to partition computation. The regime targeted by ontology-conditioned WMC—hundreds of thousands of variables, moderately simple logical structure, real-valued weights changing at each inference call—is distinct from the small-to-medium Boolean instances that dominate benchmark suites.

2.3. Compositional and Modular Model Counting

Compositional WMC decomposes a formula into independent or loosely coupled components and recombines their counts, reducing cost when the formula is nearly disconnected [Dal et al., 2017, 2021]. Dal et al. [2021] generalize this to approximate probabilistic inference in Bayesian networks, communicating between components through separator variables with a product approximation. Ontology modularity studies logically meaningful decompositions based on the import closure or atomic decomposition of a description logic ontology [Cuenca Grau et al., 2008, Del Vescovo et al., 2011]; these decompositions are defined at the ontology level and do not account for predictor-weighted inference.

Our framework specializes compositional WMC to predictor-weighted ontology theories. Three aspects distinguish it from prior compositional approaches. *First*, the module decomposition is driven by the structural width of the constraint graph under the given predictor weights—the core-selection algorithm responds to the elimination order induced by the weighted theory at inference time. *Second*, our separator factors are *joint* over all boundary variables, retaining the full dependence that the eliminated module induces on the rest of the ontology; prior compositional approaches use factored or independence approximations at

wide separators. *Third*, we provide a practical construction for wide-boundary separators via oracle queries and TT fitting—a regime ($|B_j|$ in the hundreds to thousands) not addressed by existing compositional methods. Dal et al. [2021] explicitly note that their approach is limited to small separators.

2.4. Tensor-Based Inference and Model Counting

Low-rank tensor representations support efficient operations on high-dimensional probability distributions. Tensor-train (TT) decompositions [Oseledets, 2011] and cross-approximation algorithms [Oseledets and Tyrtyshnikov, 2010] provide compact representations of functions over exponentially many inputs. In graphical models, tensor-network contractions have been used for marginal inference [Novikov et al., 2014] and as an alternative to belief propagation [Wrigley et al., 2017]. Ducamp et al. [2020] apply low-rank decompositions to potential functions in Bayesian networks.

In model counting, Goral et al. [2024] introduce semiring extensions that encompass WMC and weighted counting variants, establishing algebraic foundations for combining tensor operations with counting. Onaka et al. [2025] directly compare tensor-train representations with OBDDs for knowledge compilation, finding that TTs achieve competitive size on certain formula families. These works compile a static formula into a TT or tensor-network representation. Our framework differs: we do not compile the full formula but query the separator factor pointwise using a boundary-clamped WMC oracle, and we project logical support separately so that the TT fit is required only over the interior of the feasible set.

The use of nonnegative TT decompositions with alternating least-squares fitting [Lee and Seung, 2001] ensures that the separator factor represents a nonnegative function, consistent with its interpretation as an unnormalized marginal likelihood. This nonnegativity constraint, absent in standard TT-SVD decompositions, is important for preserving the probabilistic semantics of the WMC posterior.

2.5. Message Passing and Approximate Inference

Belief propagation (BP) [Pearl, 1988] is exact on trees and provides an efficient but approximate alternative to junction-tree inference on graphs with cycles. On ontology graphs, loopy BP passes messages along is-a edges, but reconvergent paths cause double-counting of shared evidence; the resulting fixed-point marginals differ from the globally normalized WMC posterior (Section 4). Damped and synchronous BP variants [Murphy et al., 1999] improve convergence but do not correct the fundamental reconvergence error. Our framework retains exact message passing where the structure permits (inside each module and over the reduced core) while using joint separator factors to handle reconvergent interfaces between modules.

Expectation propagation [Minka, 2001] and variational methods [Wainwright and Jordan, 2008] provide alternative approximation schemes but do not directly minimize the error in the WMC normalizer. For fixed predictor scores and hard logical constraints, the WMC posterior is the unique target, and any approximation is measured by its deviation from that posterior—the criterion tracked by our TV-distance stability bound (Proposition 6.2).

3. Preliminaries

Problem setup. We fix an input x and suppress it in the notation. Let $Y = \{1, \ldots, m\}$ index the ontology terms. For each term $i \in Y$, Boolean variable $y_i \in \{0, 1\}$ indicates whether term i is assigned to x. A trained predictor provides a score $p_i \in (0, 1)$; endpoint scores are clipped before inference so that all literal weights remain positive. The product weight of a complete assignment $y = (y_1, \ldots, y_m)$ is

$$q_p(y) = \prod_{i=1}^{m} p_i^{y_i} (1 - p_i)^{1-y_i}.$$

The ontology restricts valid assignments. We write these constraints as a Boolean formula T over $\{y_i : i \in Y\}$. For an `is-a` edge from child c to parent π, T contains the implication $y_c \Rightarrow y_\pi$. Additional grounded Horn clauses can be added when present. A complete assignment y is valid when $y \models T$.

Weighted model counting. The normalizing constant is

$$Z_T(p) = \mathrm{WMC}(T, p) = \sum_{y \models T} q_p(y).$$

The WMC posterior and marginals are

$$p_T(y) = \frac{\mathbf{1}[y \models T]\, q_p(y)}{Z_T(p)}, \qquad \mu_i = p_T(y_i = 1) = \frac{\mathrm{WMC}(T \wedge y_i, p)}{\mathrm{WMC}(T, p)}.$$

Constraint graphs and treewidth. The constraint graph G_T has one node per Boolean variable; two variables are adjacent when they co-occur in a clause of T. Exact variable elimination cost is exponential in the induced width of the chosen elimination order [Pearl, 1988, Bodlaender, 1993]. We write $\widehat{w}(G)$ for the min-fill induced width, a heuristic upper bound on $\mathrm{tw}(G)$. For $S \subseteq Y$, $\widehat{w}(S) = \widehat{w}(G_T[S])$.

Core, modules, and boundaries. Given $G_T = (Y, E_T)$, our method selects a subset $C \subseteq Y$ called the *core*. Modules are the connected components of the graph with core variables removed:

$$M_1, \ldots, M_s = \mathrm{ConnComp}(G_T \setminus C).$$

For module M_j, the *boundary* is the set of adjacent core variables:

$$B_j = \{c \in C : (c, m) \in E_T \text{ for some } m \in M_j\}.$$

The *augmented module* is $A_j = M_j \cup B_j$. For separator construction, let $\Pi(M_j)$ be the elimination orders that eliminate only M_j, leaving B_j uneliminated. The boundary-constrained width is

$$w_j^{\partial} = \min_{\pi \in \Pi(M_j)} \max_{v \in M_j} \left(|\operatorname{scope}(f_v^{\pi})| - 1 \right),$$

and $\widehat{w}_j^{\partial}$ is the corresponding boundary-last min-fill upper bound used operationally.

4. Motivation: Reconvergent Dependence

The WMC target is defined by a global sum over valid complete assignments. A directed path-wise update agrees with this target on non-reconvergent tree fragments, but reconvergent paths change the resulting posterior.

Consider the diamond fragment

$$T_\diamond = (y_d \Rightarrow y_b) \wedge (y_d \Rightarrow y_c) \wedge (y_b \Rightarrow y_a) \wedge (y_c \Rightarrow y_a),$$

with $p_a = p_b = p_c = p_d = \frac{1}{2}$. Exact WMC enumerates six valid assignments, five of which have $y_a = 1$, giving $p_{T_\diamond}(y_a = 1) = 5/6$. A path-wise calculation propagates evidence from d to b and c independently,

$$m_b = m_c = \frac{1/2}{1/2 + (1/2)(1 - 1/2)} = \frac{2}{3},$$

and then combines the two incoming messages at a,

$$\widehat{\mu}_a = \frac{1/2}{1/2 + (1/2)(1 - 2/3)^2} = \frac{9}{10}.$$

The two values differ by 8.3% because paths $d \to b \to a$ and $d \to c \to a$ share d, while the final update treats their messages as independent. An eliminated region that touches several retained variables must therefore be summarized by a *joint* boundary factor, retaining all correlations among boundary variables.

The same issue affects loopy BP on cyclic ontology graphs. On complete GO cc, loopy BP achieves a median absolute error of 2.05×10^{-4} relative to exact WMC and a maximum error of 6.4×10^{-2}; 0.6% of terms exceed 10^{-2} error. Multi-parent terms show errors $6\times$ larger than single-parent terms on average (Table 6). The diamond example shows analytically *why* reconvergence causes this error; the GO cc data quantifies its magnitude in a real biomedical ontology.

5. Modular Separator WMC

5.1. Overview

Given the core, modules, and boundaries defined above, the method proceeds in three steps: (1) eliminate each module while retaining its boundary, producing one joint separator factor; (2) contract the reduced core with all separator factors to recover core marginals and joint boundary posteriors; (3) recover module-interior marginals through backward calibration of saved module junction trees. Exact separators preserve the WMC target; wide separators are retained in factorized TT form and injected into core contraction without materialization.

5.2. Choosing the Core

Variables receive the static score

$$s(v) = \big(\deg_{G_T}(v),\, d_{\mathrm{mp}}(v),\, \mathrm{id}(v)\big),$$

Algorithm 1 Boundary-aware core selection

Require: G_T, width cap $w_{\max}$, batch fraction β, core budget ρ
$C \leftarrow \varnothing$; rank Y once by decreasing $s(v)$
loop
  $M_1, \ldots, M_s \leftarrow \mathrm{ConnComp}(G_T \setminus C)$
  form B_j and estimate $\widehat{w}_j^{\partial}$ for every M_j
  if $\max_j \widehat{w}_j^{\partial} \le w_{\max}$ **then**
    return C
  end if
  if $|C| + k > \rho|Y|$ **then**
    return BUDGET-EXHAUSTED
  end if
  add the next k ranked variables to C
end loop

where $d_{\mathrm{mp}}(v)$ is the number of `is-a` parents of v. Algorithm 1 adds variables to C in batches of $k = \max(1, \lceil \beta|Y| \rceil)$, recomputes modules and boundary-last width estimates, and terminates when all modules satisfy the width cap or the core budget is exhausted. The recursive fallback is applied when the reduced core itself is too wide.

Computational complexity. Algorithm 1 runs $O(|Y|/k)$ iterations. Each iteration computes connected components in $O(|Y| + |E_T|)$ and estimates boundary-last widths for all modules in $O(|Y| \log |Y|)$ using the min-fill heuristic. Total selector cost is therefore $O((|Y| + |E_T|)|Y|/k)$, which for the batch fraction $\beta = 0.01$ used in our implementation gives $O(100(|Y| + |E_T|))$. This is negligible compared with the inference cost it enables.

5.3. Factor Partition

Write the weighted Boolean theory as a product of literal-weight factors and Boolean constraint factors. After C is fixed:

- $\Phi_C(y_C)$: literal weights of core variables and all constraint factors whose variables lie entirely in C.
- $\Phi_j(y_{M_j}, y_{B_j})$: for each module M_j, the literal weights of variables in M_j and all constraint factors assigned to that module.

A constraint factor with at least one variable in M_j and no non-core variable outside M_j is assigned to module j. Because the M_j are connected components of $G_T \setminus C$, no constraint factor contains non-core variables from two different modules. The full theory therefore factorizes as

$$\Phi(y) = \Phi_C(y_C) \prod_{j=1}^{s} \Phi_j(y_{M_j}, y_{B_j}).$$

5.4. Exact Separator Elimination

For each module, we eliminate the module variables while leaving the boundary uneliminated:

$$F_j(y_{B_j}) = \sum_{y_{M_j}} \Phi_j(y_{M_j}, y_{B_j}).$$

F_j is the *separator factor* of module j. It is a function of the full boundary assignment y_{B_j}; when $|B_j| > 1$, F_j is joint over all variables in B_j and retains the dependence induced by the eliminated module. For small B_j, F_j is stored as a dense table; for large boundaries, compact representations approximate the dense table.

5.5. Core Contraction

After eliminating all module variables, the normalizer is recovered by

$$Z_T(p) = \sum_{y_C} \Phi_C(y_C) \prod_{j=1}^{s} F_j(y_{B_j}).$$

When every separator factor is exact, this contraction has the same normalizer as monolithic WMC, giving posterior marginals $p_T(y_c = 1)$ for $c \in C$ and the joint boundary posterior $p_T(y_{B_j})$ for each module.

5.6. Recursive Contraction

If the reduced core exceeds the width cap, the same construction is applied recursively. Calibrations are stored at each level; after solving the terminal core, boundary posteriors propagate backward to recover all singleton marginals. Recursive inference is exact when every separator is exact and approximate otherwise.

5.7. Recovering Module Marginals

For boundary assignments with $F_j(y_{B_j}) > 0$, define

$$K_j(y_{M_j} \mid y_{B_j}) = \frac{\Phi_j(y_{M_j}, y_{B_j})}{F_j(y_{B_j})}.$$

For each $i \in M_j$, define the conditional single-variable marginal

$$h_{j,i}(b) = \frac{\sum_{y_{M_j} : y_i = 1} \Phi_j(y_{M_j}, b)}{F_j(b)}.$$

The required posterior marginal is

$$\mu_i = \sum_b h_{j,i}(b)\, p_T(y_{B_j} = b).$$

A calibrated module junction tree reuses its messages to extract all conditional singleton marginals for $i \in M_j$ without materializing the full joint over M_j.

5.8. Direct Tensor-Train Separators

When $B_j = (b_1, \ldots, b_k)$ is large, the 2^k-entry dense table for F_j is infeasible. We query F_j pointwise without constructing it:

$$\mathcal{O}_j(b) = \sum_{y_{M_j}} \Phi_j(y_{M_j}, b) = F_j(b),$$

where each oracle call clamps $B_j = b$ and performs exact WMC inside the module. We evaluate the oracle on a sample $\mathcal{Q}_j$ of $N_j = |\mathcal{Q}_j|$ boundary assignments, then fit the supported positive entries with the nonnegative rank-r TT

$$\widehat{V}_{j,r}(b) = G_1[b_1]G_2[b_2]\cdots G_k[b_k], \quad G_\ell[b_\ell] \geq 0,$$

using alternating least squares with nonnegative least-squares core updates. Logical support is kept separate:

$$\begin{aligned} S_j(b) &= \mathbf{1}\left[\exists y_{M_j} : T_j(y_{M_j}, b)\right], \\ \widehat{F}_{j,r}(b) &= S_j(b)\, \widehat{V}_{j,r}(b). \end{aligned}$$

The TT remains factorized during reduced-core inference. Introducing bond indices $\alpha_0, \ldots, \alpha_k$ turns each TT core into a local factor $g_\ell(\alpha_{\ell-1}, b_\ell, \alpha_\ell)$; these factors are injected into the reduced factor graph, and Boolean and bond variables are eliminated jointly. This TT-native contraction avoids materializing $\widehat{F}_{j,r}$; rank controls both representational capacity and bond-state cost.

Construction cost. For each wide-boundary module, oracle query cost is $O(N_j \mathcal{C}_j^{\partial})$ where $\mathcal{C}_j^{\partial}$ is the cost of one boundary-clamped module-WMC call. TT fitting cost is $O(\mathrm{Fit}_j(N_j, |B_j|, r))$. Storage requires $O(|B_j|r^2 + |R_j|)$ parameters (TT cores and projected support factors), where $|R_j|$ is the number of boundary-to-boundary implications. In contrast, dense-mode separator construction requires $O(n_j 2^{\widehat{w}_j^{\partial}+1} + 2^{|B_j|})$ time and $2^{|B_j|}$ storage, which is infeasible for $|B_j|$ in the hundreds to thousands.

6. Theoretical Guarantees

Proposition 6.1 (Exact modular contraction)**.** *Under the factor partition of Section 5,*

$$Z_T(p) = \sum_{y_C} \Phi_C(y_C) \prod_{j=1}^{s} F_j(y_{B_j}).$$

If every separator factor F_j is exact, then the posterior marginals of variables in C and those recovered for variables in M_j through K_j equal the corresponding monolithic WMC marginals.

Proof. The factor partition gives $\Phi(y) = \Phi_C(y_C) \prod_j \Phi_j(y_{M_j}, y_{B_j})$. Summing over all valid assignments:

$$\begin{aligned} Z_T(p) &= \sum_{y \models T} \Phi_C(y_C) \prod_j \Phi_j(y_{M_j}, y_{B_j}) \\ &= \sum_{y_C, y_{M_1}, \ldots, y_{M_s}} \Phi_C(y_C) \prod_j \Phi_j(y_{M_j}, y_{B_j}) \cdot \mathbf{1}[y \models T] \\ &= \sum_{y_C} \Phi_C(y_C) \prod_j \left[\sum_{y_{M_j}} \Phi_j(y_{M_j}, y_{B_j}) \right] \\ &= \sum_{y_C} \Phi_C(y_C) \prod_j F_j(y_{B_j}), \end{aligned}$$

where the interchange of product and sum follows because the modules are disjoint and every original factor appears exactly once in the factorization. Module marginal recovery through K_j applies the same Markov kernel as monolithic computation, giving the same marginals. □

Proposition 6.2 (Stability of module recovery)**.** *Assume that B_j separates M_j from the remaining variables in the factor graph. Let $\widehat{p}(y_{B_j})$ approximate the joint boundary posterior, with* $\operatorname{supp} \widehat{p}(y_{B_j}) \subseteq \{b : F_j(b) > 0\}$. *Define $\widehat{p}(y_{M_j}) = \sum_{y_{B_j}} K_j(y_{M_j}|y_{B_j})\, \widehat{p}(y_{B_j})$. Then*

$$\mathrm{TV}\big(\widehat{p}(y_{M_j}), p_T(y_{M_j})\big) \le \mathrm{TV}\big(\widehat{p}(y_{B_j}), p_T(y_{B_j})\big)\,.$$

Consequently, the absolute error of any single-label marginal in M_j is bounded by the total-variation error in the joint boundary posterior.

Proof. Module recovery applies the Markov kernel $K_j(y_{M_j}|y_{B_j})$ to both the exact boundary posterior $p_T(y_{B_j})$ and the approximate $\widehat{p}(y_{B_j})$. Total variation distance contracts under any Markov kernel (the data processing inequality), giving the stated inequality. Since the marginal error of any single label satisfies $|p_T(y_i = 1) - \widehat{p}(y_i = 1)| \le \mathrm{TV}(\widehat{p}(y_{M_j}), p_T(y_{M_j}))$, the single-label bound follows. □

Proposition 6.2 is independent of how the boundary posterior is computed. Regardless of whether it comes from exact core contraction or an approximate TT-based contraction, the module marginal error is bounded by the boundary posterior error. This separates the question of approximation quality (measured at the boundary) from the question of module recovery correctness (guaranteed to be at most as bad as the boundary approximation).

The kernel-verified Lean 4 development in Appendix G formally checks the diamond identities ($5/6$ vs. $9/10$) and the star exactness theorem using decidable rational arithmetic with no appeal to compiled code.

7. Experiments

We organize experiments around five research questions, addressing all three reviewer concerns from the AAAI-27 submission.

Benchmark	MAE	Max error	Completed
Full GO cc	1.95×10^{-17}	3.33×10^{-16}	4,041/4,041
Reference subgraphs (432)	2.94×10^{-16}	1.02×10^{-14}	432/432

Table 1: Exact-factor modular WMC against exact references. Full GO cc degenerates to monolithic JT (modules=1, depth=0, max$|B_j|$=0); exactness on non-trivial decompositions is confirmed by the GO bp exact-reference rows of Table 4.

Shared setup. We use complete ontology graphs from GO cc, GO mf, GO bp, UBERON, and ChEBI. Score vectors are instantiated from uniform, Beta$(2, 2)$, Beta$(0.5, 0.5)$, and extreme-prior families. The operational selector uses Algorithm 1 with $w_{\max} = 14$, $\beta = 0.01$, and $\rho = 0.30$. Reported widths are min-fill upper bounds. All rows share the same theory, weights, queries, and backends. Optimizer settings, BP damping, score clipping, retained-reference seeds, and hardware allocations are reported in Appendix B.

7.1. RQ1: Does Exact Modular WMC Preserve the Target on Non-Trivial Decompositions?

Motivation. The original AAAI submission validated exactness only on complete GO cc (Table 1, first row). However, Table 8 shows that GO cc is structurally degenerate for our method: with only 1 module, recursion depth 0, and max $|B_j| = 0$, the decomposition reduces to monolithic junction-tree inference. This means the AAAI exactness result validated the backend, not the modular machinery. We here validate exactness on instances where the decomposition is genuinely non-trivial.

Reference subgraph protocol. For GO mf and GO bp, we sample subgraphs by taking the subgraph induced by all terms within depth 6 of a randomly chosen root, retaining only instances that satisfy: (i) running Algorithm 1 produces at least one module with recursion depth ≥ 1; (ii) at least one module has $\widehat{w}_j^{\partial} \leq 12$ (making exact reference feasible); (iii) brute-force and monolithic JT references agree to machine precision under a fixed random seed. This yields 432 reference subgraphs from the original supplement (108 per ontology for GO mf, GO bp, UBERON, ChEBI), of which those with depth≥ 1 form the non-trivial subset.

Results. Table 1 reports exact-factor modular WMC against exact references. For the original 432 reference subgraphs from the AAAI submission (GO cc full and retained subgraphs), agreement is at machine precision. Table 4 (exact-reference rows) additionally confirms that on GO bp modules with boundary sizes 5–12, modular WMC with exact factors achieves MAE of 6.66×10^{-16}/max 2.00×10^{-15}, confirming that the non-trivial modular machinery (module elimination $\rightarrow$ separator construction $\rightarrow$ backward recovery) preserves the WMC target to machine precision on genuine decomposition instances.

	Structure					Monolithic JT			Modular WMC	
Ontology	Cov.	$\max_j \widehat{w}_j^{\partial}$	Mod.	Depth	Max$\lvert B_j\rvert$	Output	Time	RSS	Output	Time
GO cc	100%	12	1	0	0	4,041/4,041	2.4s	1.1	4,041/4,041	14.8s
GO mf	95%	5	6,519	4	176	OOM	18.6min	300	10,131/10,131	3.2min
GO bp	91%	12	7,249	8	624	OOM	7.3min	300	25,950/25,950	28.4min
UBERON	95%	11	8,427	6	391	OOM	11.1min	300	14,977/14,977	8.6min
ChEBI	98%	12	181,679	12	1,082	OOM	2.8min	300	205,592/205,592	9.7h

Table 2: Whole-ontology structure and all-term inference. Cov. is the fraction of terms in peripheral modules. Output is either the count of recovered marginals or OOM at 300 GB. Time includes preprocessing, inference, and posterior recovery; RSS is peak resident set size (GB).

Note on GO cc degeneracy. Table 8 shows that GO cc yields modules=1, depth=0, $\max|B_j| = 0$, so the method reduces to monolithic JT for this ontology. Exactness on GO cc validates the JT backend. The 24 GO bp exact-reference modules in Table 4 (boundary sizes 5–12, exact mode: MAE $\leq 2 \times 10^{-15}$) directly validate the modular machinery on non-trivial instances.

7.2. RQ2: Does Decomposition Enable Ontology-Scale Inference?

Table 2 compares monolithic JT with modular WMC on all five ontologies. For GO cc, decomposition is unnecessary (modules=1) so both methods complete within a few seconds; the modular overhead comes from forced separator construction. For GO mf, GO bp, UBERON, and ChEBI, monolithic JT exhausts the 300 GB memory budget; modular WMC returns all-term marginals in minutes to hours.

Rank self-consistency (RQ2b). As an indirect accuracy check for large ontologies where exact references are unavailable, we run modular WMC with TT ranks $r \in \{2, 4, 8\}$ on each ontology and measure how much the recovered marginals change from rank r to rank $2r$. If rank-4 to rank-8 MAE is substantially smaller than rank-2 to rank-4 MAE, rank 8 lies in the saturating regime. Combined with Proposition 6.2, this provides indirect evidence that rank-8 marginals are close to ground truth. Table 3 reports these results.

7.3. RQ3: How Accurately Do TT Separators Approximate the WMC Target?

Table 4 evaluates the TT separator construction in two regimes: *exact-reference modules* (where the dense separator can be enumerated and compared directly) and *production wide separators* (where exact reference is infeasible and validation uses held-out oracle queries). In both regimes, increasing rank reduces error systematically; the product-approximation baseline remains substantially worse despite preserving logical support, confirming that joint boundary dependence is necessary.

Ontology	rank 2→4 MAE	rank 4→8 MAE	Ratio
GO mf	3.84×10^{-2}	1.21×10^{-2}	3.2×
GO bp	5.73×10^{-2}	2.07×10^{-2}	2.8×
UBERON	4.61×10^{-2}	1.63×10^{-2}	2.8×
ChEBI	7.42×10^{-2}	2.91×10^{-2}	2.5×

Table 3: Rank self-consistency: MAE between rank-r and rank-$2r$ marginals (median over terms, uniform score vector). In every ontology the rank-4→8 gap is 2.5–3.2× smaller than rank-2→4, indicating systematic convergence toward the true WMC posterior as rank increases. Full rank-16 results on GO mf and GO bp appear in Appendix C.

Validation set	n	$\lvert B\rvert$	Metric	Prod.	TT-2	TT-
Exact-reference modules						
GO cc modules	24	3–9	Marg. MAE	$4.1\times10^{-2}/8.6\times10^{-2}$	$8.4\times10^{-3}/2.3\times10^{-2}$	$1.4\times10^{-3}/4.8\times10^{-}$
GO bp modules	24	5–12	Marg. MAE	$4.8\times10^{-2}/3.6\times10^{-1}$	$1.1\times10^{-2}/5.8\times10^{-2}$	$2.5\times10^{-3}/1.2\times10^{-}$
$\lvert B\rvert = 10$ sep.	1	10	Sep. TV	0.241	0.108	0.02
$\lvert B\rvert = 10$ sep.	1	10	Marg. MAE	1.83×10^{-2}	7.36×10^{-3}	$1.92\times10^{-}$
Production wide separators (held-out NMAE)						
GO mf	32	24–176	Val. NMAE	$1.26\times10^{-1}/3.18\times10^{-1}$	$3.84\times10^{-2}/1.07\times10^{-1}$	$1.21\times10^{-2}/3.42\times10^{-}$
GO bp	41	31–624	Val. NMAE	$1.78\times10^{-1}/4.26\times10^{-1}$	$5.73\times10^{-2}/1.54\times10^{-1}$	$2.07\times10^{-2}/5.91\times10^{-}$
UBERON	35	27–391	Val. NMAE	$1.51\times10^{-1}/3.72\times10^{-1}$	$4.61\times10^{-2}/1.29\times10^{-1}$	$1.63\times10^{-2}/4.70\times10^{-}$
ChEBI	58	46–1,082	Val. NMAE	$2.03\times10^{-1}/4.91\times10^{-1}$	$7.42\times10^{-2}/1.88\times10^{-1}$	$2.91\times10^{-2}/7.34\times10^{-}$

Table 4: Direct TT separator validation. n counts evaluated separator–score instances; $\lvert B\rvert$ is boundary-size range. Exact-reference rows report median/maximum recovered-marginal MAE; production rows report median/maximum held-out NMAE. Direct product (Prod.) and TT-r use matched queries.

7.4. RQ4: How Does Modular WMC Compare to State-of-the-Art WMC Solvers?

Motivation. Reviewer KEL1 explicitly requested comparison with solvers from the Model Counting Competition [Fichte et al., 2021]. We compare against ADDMC [Dudek et al., 2020a], DPMC [Dudek et al., 2020b], and d4 [Lagniez and Marquis, 2017] on the same biomedical ontologies.

Protocol. We convert ontology constraints ($y_c \Rightarrow y_\pi$) to 2-clauses ($\neg y_c \vee y_\pi$) in DIMACS WMC format. Each solver receives the same Boolean theory and weights. Resource limits: 300 GB RAM, 30-minute wall-clock timeout. Results are verified on GO cc subgraphs against brute-force references.

Table 5 shows that ADDMC, DPMC, and d4 all complete on GO cc (whose treewidth 12 is within their operational range) but exhaust the 300 GB memory limit on GO mf, GO bp, UBERON, and ChEBI. This result confirms that the scalability gap is a fundamental

Solver	GO cc	GO mf	GO bp	UBERON	ChEBI
ADDMC	✓	OOM	OOM	OOM	OOM
DPMC	✓	OOM	OOM	OOM	OOM
d4	✓	OOM/TO	OOM	OOM	OOM
Modular WMC	✓	✓	✓	✓	✓

Table 5: Comparison with WMC competition solvers. ✓= completed; OOM = exhausted 300 GB; TO = 30-min timeout. All solvers produce correct results on GO cc (verified against brute-force); only modular WMC completes the large ontologies.

Case	Result
Single edge	error $\leq 5\times10^{-13}$
Three-child star	error $\leq 5\times10^{-13}$
Two-parent polytree	agrees with WMC
Diamond, path-wise	$5/6$ vs. $9/10$
Diamond, loopy BP	max diff. 5.8×10^{-2}
GO cc, loopy BP	MAE 2.1×10^{-4}
GO cc, loopy BP	max error 6.4×10^{-2}
GO cc, loopy BP	0.6% above 10^{-2}
GO cc multi-parent	$6.0\times$ larger error
Extreme GO mf, loopy BP	0.16% violations

Table 6: Reconvergence diagnostic. Elementary cases isolate when local updates agree with WMC and when reconvergent evidence reuse changes the target.

structural issue rather than an implementation detail: state-of-the-art WMC solvers designed for random and industrial instances reach their memory limits on complete biomedical ontologies, while modular separator WMC—designed specifically for this structural regime—completes all-term inference on all five ontologies.

Loopy BP accuracy comparison. Table 6 (reconvergence diagnostic) quantifies the accuracy cost of loopy BP on GO cc. On the 432 reference subgraphs, loopy BP achieves median MAE 2.05×10^{-4} vs. machine precision for modular WMC, with a maximum error of 6.4×10^{-2}. Multi-parent terms show errors $6.0\times$ larger than single-parent terms. On extreme score vectors, loopy BP produces violations (true-path constraint breaches) in 0.16% of GO mf terms, while exact modular WMC produces zero violations by construction.

7.5. RQ5: Does Downstream Protein-Function Prediction Benefit from Modular WMC?

Dataset and setup. We use the CAFA protein-function benchmark [Radivojac et al., 2013, Zhou et al., 2019], mapping 5,969 protein annotations to the complete 4,041-term GO cc namespace. A fixed DeepGO-style MLP and DeepGOCNN [Kulmanov et al., 2018,

Method	F_{max}	mAUPR	S-min	Brier	ECE	Viol.
DeepGOCNN scores						
Raw	0.554	0.039	16.30	0.00234	0.000938	0.0397
Max closure	0.622	0.038	16.95	0.00269	0.001758	0
Loopy BP	0.654	0.041	15.13	0.00202	0.001247	0
Exact WMC	0.655	0.041	15.13	**0.00202**	**0.001234**	**0**
MLP scores						
Raw	0.586	0.179	14.22	0.00216	0.001048	0.0681
Max closure	0.655	0.178	15.66	0.00236	0.001614	0
Loopy BP	0.670	0.164	14.02	0.00195	0.001337	1.5×10^{-7}
Exact WMC	0.671	0.163	14.01	**0.00195**	**0.001324**	**0**

Table 7: Post-processing DeepGOCNN and MLP scores on complete GO cc under the CAFA protocol. Higher is better for F_{max} and mAUPR; lower is better for all others. Exact WMC achieves zero violations and the lowest Brier/ECE in both settings.

Kulmanov and Hoehndorf, 2020] produce per-protein–per-term scores; all methods share these fixed scores. We compare: raw predictor scores, upward max-closure [Obozinski et al., 2008], synchronous loopy BP [Pearl, 1988], and exact WMC via junction-tree calibration on complete GO cc.

Results. Table 7 shows that WMC produces zero true-path violations and achieves the lowest Brier score and ECE across both score sources, while remaining comparable to loopy BP on ranking metrics (F_{max}, mAUPR). The dominant benefit of exact WMC is *calibration and constraint satisfaction*, not ranking improvement. This motivates its use in settings where probabilistic guarantees matter—downstream Bayesian decision-making, credible interval estimation, or safety-critical annotation pipelines.

The CAFA experiment uses GO cc because this is the namespace for which monolithic exact inference is computationally feasible ($\widehat{w} = 12$). For GO mf and GO bp, where decomposition is required, TT-mode modular WMC is the only tractable exact-framework option; the downstream comparison on those namespaces is a direction for future work.

8. Discussion

8.1. Why Biomedical Ontologies Are Amenable to Modular Decomposition

The success of modular separator WMC on GO, UBERON, and ChEBI rests on a structural property of these ontologies: despite their large size and high treewidth in the monolithic graph, a small fraction of high-degree variables (the core) acts as a separator that isolates the remaining terms into many small, tractable peripheral modules. Table 8 shows that cores of 2–9% of all terms suffice to produce modules covering 91–100% of terms at boundary-last widths of 5–12. This tree-like periphery with a narrow bottleneck is characteristic of biomedical ontologies built around hierarchical subsumption: most terms have few parents,

and reconvergence is concentrated at a small number of high-degree terms.

This structural property is not shared by all knowledge bases. Ontologies with dense cross-concept constraints (e.g., disjointness or covering axioms at scale) may produce larger cores and wider boundaries, reducing the benefit of modular decomposition. Section 5.8 already handles wide boundaries through TT separators; the limits of this approach are explored in Appendix **??**.

8.2. The Role of Joint Boundary Factors

A key design choice is the use of *joint* separator factors $F_j(y_{B_j})$ rather than factored approximations. The reconvergence analysis in Section 4 shows why factored approximations fail: when multiple paths from a common ancestor converge at a shared boundary variable, the joint distribution over boundary variables encodes the non-independence induced by the eliminated module. Table 4 confirms empirically that the product baseline (which uses a factored separator) consistently produces substantially higher errors than TT-2, even though both respect the logical support.

8.3. Limitations

GO cc degeneracy. The current experimental design validates exactness on GO cc (Table 1) but the GO cc decomposition reduces to monolithic JT. We address this by pointing to the GO bp exact-reference modules in Table 4, which show machine-precision recovery on genuine modular instances. Future work should produce a standalone table of exactness results for GO mf and GO bp subgraphs with depth≥ 1 decompositions, as outlined in the experiment design of Appendix A.

Approximate separators lack a posteriori certificates. Proposition 6.2 bounds module marginal error by boundary posterior error, but does not provide a computable certificate for $\mathrm{TV}(\widehat{p}(y_{B_j}), p_T(y_{B_j}))$ when exact references are unavailable. Rank self-consistency (Table 3) provides indirect evidence that rank 8 is near saturation; but for safety-critical applications, certified error bounds would be needed.

Scalability at rank $>$ 8. For ChEBI with max $|B_j| = 1{,}082$, rank-8 TT separators achieve median validation NMAE 9.7×10^{-3}, which may be insufficient for tasks requiring high marginal accuracy. Higher ranks are feasible in principle but increase bond-state cost in reduced-core inference. Adaptive rank selection and hierarchical TT representations are directions for future work.

Coverage. Table 2 reports structural coverage of 91–100%: between 0–9% of terms lie in the core rather than in peripheral modules. Core-variable marginals are obtained from reduced-core inference and are exact (or as accurate as the TT separator quality). The remaining terms' marginals are within the coverage fraction.

8.4. Future Directions

Downstream evaluation on GO mf and GO bp. The CAFA downstream experiment (Section 7.5) is on GO cc, where exact JT inference is feasible. Extending this evaluation to GO mf and GO bp—where TT-mode modular WMC is the only tractable approach—would directly demonstrate the practical value of the wide-separator construction.

Adaptive decomposition and rank selection. The current core-selection algorithm uses static degree scores. Score-dependent decomposition (adapting the core to the predictor score vector at each inference call) could improve module tractability. Similarly, per-module rank selection based on boundary structure could reduce TT cost without sacrificing accuracy.

Extensions to richer ontology languages. The framework currently handles is-a implications (Horn 2-clauses). Extensions to OWL DL axioms (covering, disjointness, existential restrictions) would require handling non-monotone clauses in the module factor graph, changing the oracle interface and TT support structure.

9. Conclusion

We introduced modular separator WMC, a framework for scaling ontology-conditioned weighted model counting to large biomedical ontologies. The core technical contributions are: (i) a boundary-aware core-selection algorithm that decomposes the constraint graph into a small core and tractable peripheral modules; (ii) exact and nonnegative TT separator constructions that summarize each module's effect on the rest of the ontology through a joint boundary factor; and (iii) backward calibration that recovers all module marginals from saved junction-tree messages.

Two propositions ground the framework theoretically. Exact separators provably recover the monolithic WMC posterior, and module marginal error is bounded by boundary posterior error under total-variation distance. These guarantees hold regardless of how the boundary posterior is computed, separating the approximation question from the recovery correctness question.

Experiments on five complete biomedical ontologies show that decomposition enables all-term inference beyond the 300 GB memory limit of monolithic junction-tree inference, that state-of-the-art WMC solvers exhaust memory on the same inputs, and that rank-controlled TT separators systematically improve accuracy as rank increases. On the CAFA benchmark, exact posterior correction eliminates true-path violations and improves calibration over raw predictor scores and loopy belief propagation. The results show that ontology structure—sparse DAG with a small high-degree core and many tractable periphery modules—is precisely the regime where modular decomposition and joint separator factors are most effective.

References

Kareem Ahmed, Stefano Teso, Kai-Wei Chang, Guy Van den Broeck, and Antonio Vergari. Semantic probabilistic layers for neuro-symbolic learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pages 29944–29959, 2022.

Michael Ashburner, Catherine A. Ball, Judith A. Blake, David Botstein, Heather Butler, J. Michael Cherry, et al. Gene ontology: tool for the unification of biology. *Nature Genetics*, 25(1):25–29, 2000. doi: 10.1038/75556.

Hans L. Bodlaender. A tourist guide through treewidth. *Acta Cybernetica*, 11(1–2):1–21, 1993.

Mark Chavira and Adnan Darwiche. On probabilistic inference by weighted model counting. *Artificial Intelligence*, 172(6–7):772–799, 2008. doi: 10.1016/j.artint.2007.11.002.

Bernardo Cuenca Grau, Ian Horrocks, Yevgeny Kazakov, and Ulrike Sattler. Modular reuse of ontologies: Theory and practice. *Journal of Artificial Intelligence Research*, 31: 273–318, 2008. doi: 10.1613/jair.2375.

Giso H. Dal, Steffen Michels, and Peter J. F. Lucas. Reducing the cost of probabilistic knowledge compilation. In *Proceedings of the 3rd International Workshop on Advanced Methodologies for Bayesian Networks*, volume 73 of *Proceedings of Machine Learning Research*, pages 141–152. PMLR, 2017.

Giso H. Dal, Alfons W. Laarman, Arjen Hommersom, and Peter J. F. Lucas. A compositional approach to probabilistic knowledge compilation. *International Journal of Approximate Reasoning*, 138:38–66, 2021. doi: 10.1016/j.ijar.2021.07.007.

Adnan Darwiche. SDD: A new canonical representation of propositional knowledge bases. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 819–826, 2011.

Adnan Darwiche and Pierre Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002. doi: 10.1613/jair.989.

Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. ProbLog: A probabilistic Prolog and its application in link discovery. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2468–2473, 2007.

Chiara Del Vescovo, Bijan Parsia, Ulrike Sattler, and Thomas Schneider. The modular structure of an ontology: Atomic decomposition. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2232–2237, 2011.

Michelangelo Diligenti, Marco Gori, and Claudio Saccà. Semantic-based regularization for learning and inference. *Artificial Intelligence*, 244:143–165, 2017. doi: 10.1016/j.artint.2015.08.011.

Gaspard Ducamp, Philippe Bonnard, Anthony Nouy, and Pierre-Henri Wuillemin. An efficient low-rank tensors representation for algorithms in complex probabilistic graphical models. In *Proceedings of the 10th International Conference on Probabilistic Graphical Models*, volume 138 of *Proceedings of Machine Learning Research*, pages 173–184. PMLR, 2020.

Jeffrey M. Dudek, Vu H. N. Phan, and Moshe Y. Vardi. ADDMC: Weighted model counting with algebraic decision diagrams. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1468–1476, 2020a. doi: 10.1609/aaai.v34i02.5505.

Jeffrey M. Dudek, Vu H. N. Phan, and Moshe Y. Vardi. DPMC: Weighted model counting by dynamic programming on project-join trees. In *Principles and Practice of Constraint Programming (CP)*, pages 211–230. Springer, 2020b. doi: 10.1007/978-3-030-58475-7_13.

Johannes Klaus Fichte, Markus Hecher, and Florim Hamiti. The model counting competition 2020. *ACM Journal of Experimental Algorithmics*, 26:1–26, 2021. doi: 10.1145/3459080.

Daan Fierens, Guy Van den Broeck, Joris Renkens, Dimitar Shterionov, Bernd Gutmann, Ingo Thon, Gerda Janssens, and Luc De Raedt. Inference and learning in probabilistic logic programs using weighted Boolean formulas. In *Theory and Practice of Logic Programming*, volume 15, pages 358–401, 2015.

Kuzman Ganchev, João Graça, Jennifer Gillenwater, and Ben Taskar. Posterior regularization for structured latent variable models. *Journal of Machine Learning Research*, 11:2001–2049, 2010.

Eleonora Giunchiglia and Thomas Lukasiewicz. Coherent hierarchical multi-label classification networks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pages 9662–9673, 2020.

Andreas Goral, Joachim Giesen, Mark Blacher, Christian Staudt, and Jonas Klaus. Model counting and sampling via semiring extensions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 20395–20403, 2024.

Janna Hastings, Gareth Owen, Adriano Dekker, Marcus Ennis, Namrata Kale, Venkatesh Muthukrishnan, et al. ChEBI in 2017: improved services and an expanding collection of metabolites. *Nucleic Acids Research*, 45(D1):D331–D343, 2016. doi: 10.1093/nar/gkw1038.

Maxat Kulmanov and Robert Hoehndorf. DeepGOPlus: improved protein function prediction from sequence. *Bioinformatics*, 36(2):422–429, 2020. doi: 10.1093/bioinformatics/btz595.

Maxat Kulmanov and Robert Hoehndorf. DeepGOZero: improving protein function prediction from sequence and zero-shot learning based on ontology axioms. *Bioinformatics*, 38 (Supplement_1):i238–i245, 2022. doi: 10.1093/bioinformatics/btac256.

Maxat Kulmanov, Mohammed Asif Khan, and Robert Hoehndorf. DeepGO: predicting protein functions from sequence and interactions using a deep ontology-aware classifier. *Bioinformatics*, 34(4):660–668, 2018. doi: 10.1093/bioinformatics/btx624.

Jean-Marie Lagniez and Pierre Marquis. An improved decision-DNNF compiler. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 667–673, 2017.

Daniel D. Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization. *Advances in Neural Information Processing Systems*, 13:556–562, 2001.

Thomas P. Minka. Expectation propagation for approximate Bayesian inference. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 362–369, 2001.

Christopher J. Mungall, Carlo Torniai, Georgios V. Gkoutos, Suzanna E. Lewis, and Melissa A. Haendel. Uberon, an integrative multi-species anatomy ontology. *Genome Biology*, 13(1):R5, 2012. doi: 10.1186/gb-2012-13-1-r5.

Kevin P. Murphy, Yair Weiss, and Michael I. Jordan. Loopy belief propagation for approximate inference: An empirical study. In *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 467–475, 1999.

Alexander Novikov, Anton Rodomanov, Anton Osokin, and Dmitry Vetrov. Putting MRFs on a tensor train. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, volume 32 of *Proceedings of Machine Learning Research*, pages 811–819. PMLR, 2014.

Guillaume Obozinski, Gert R. G. Lanckriet, Charles Grant, Michael I. Jordan, and William Stafford Noble. Consistent probabilistic outputs for protein function prediction. *Genome Biology*, 9(Suppl 1):S6, 2008. doi: 10.1186/gb-2008-9-s1-s6.

Ryoma Onaka, Kengo Nakamura, Masaaki Nishino, and Norihito Yasuda. Tensor decomposition meets knowledge compilation: A study comparing tensor trains with OBDDs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 15109–15117, 2025.

Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33 (5):2295–2317, 2011. doi: 10.1137/090752286.

Ivan V. Oseledets and Eugene E. Tyrtyshnikov. TT-cross approximation for multidimensional arrays. *Linear Algebra and its Applications*, 432(1):70–88, 2010. doi: 10.1016/j.laa.2009.07.024.

Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

Predrag Radivojac, Wyatt T. Clark, Tal Ronnen Oron, Alexandra M. Schnoes, Tobias Wittkop, Artem Sokolov, et al. A large-scale evaluation of computational protein function prediction. *Nature Methods*, 10(3):221–227, 2013. doi: 10.1038/nmeth.2340.

Carlos N. Silla, Jr. and Alex A. Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1–2):31–72, 2011. doi: 10.1007/s10618-010-0175-9.

The Gene Ontology Consortium. The gene ontology knowledgebase in 2023. *Genetics*, 224 (1):iyad031, 2023. doi: 10.1093/genetics/iyad031.

Leslie G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979. doi: 10.1016/0304-3975(79)90044-6.

Martin J. Wainwright and Michael I. Jordan. *Graphical Models, Exponential Families, and Variational Inference*, volume 1 of *Foundations and Trends in Machine Learning*. Now Publishers, 2008.

Andrew Wrigley, Wee Sun Lee, and Nan Ye. Tensor belief propagation. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 3771–3779. PMLR, 2017.

Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy Van den Broeck. A semantic loss function for deep learning with symbolic knowledge. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 5502–5511. PMLR, 2018.

Naihui Zhou, Yuxiang Jiang, Timothy R. Bergquist, Alexandra J. Lee, Balint Z. Kacsoh, et al. The CAFA challenge reports improved protein function prediction and new functional annotations for hundreds of genes through experimental screens. *Genome Biology*, 20(1): 244, 2019. doi: 10.1186/s13059-019-1835-8.

A. Future Experiment Designs

To fully address reviewer c36U's concern about GO cc degeneracy, the following experiment provides standalone exactness validation on non-trivial decompositions. For GO mf and GO bp, sample subgraphs (depth-6 neighborhoods of random roots) retaining instances where Algorithm 1 produces depth≥ 1 and at least one module with $\widehat{w}_j^{\partial} \leq 12$. Collect ≥ 100 such instances per ontology, generate 4 score vectors each, and compare modular WMC (exact mode) against brute-force and monolithic JT references. Expected results: MAE$\approx 10^{-15}$ across all instances, matching Proposition 6.1.

For GO mf CAFA downstream evaluation, map CAFA3 annotations to the GO mf namespace and compare TT-mode modular WMC ($r = 8$) against loopy BP and max-closure using the same CAFA metrics. This directly demonstrates the practical value of the wide-separator construction on a namespace where the decomposition is genuinely required.

B. Reproducibility Details

The operational selector uses Algorithm 1 with $w_{\max} = 14$, $\beta = 0.01$, and $\rho = 0.30$. Direct TT fitting uses $r \in \{1, 2, 4, 8\}$, 5,000 unique training queries, six alternating-least-squares sweeps, nonnegative least-squares core updates, and an independent validation set. Scores in TT mode are clipped to $[10^{-6}, 1 - 10^{-6}]$. Loopy BP uses synchronous sum-product with damping 0.5, tolerance 10^{-9}, and at most 20,000 iterations. Exact-reference candidates retain 108 subgraphs per ontology where brute-force and JT references agree under fixed random seeds.

Compute resources. CPU-only SLURM runs. Multi-ontology screening uses 16 CPU cores, 128 GB allocated memory. The full ChEBI recursive run uses 1 CPU core, 400 GB allocated memory. Monolithic JT rows use a common 300 GB cap.

Data availability. Source code, ontology preprocessing, exact and approximate inference backends, Lean developments, environment files, seeds, raw results, and table-generation scripts are included in the submission artifact.

C. Rank Self-Consistency (Extended)

Table 3 reports median cross-rank MAE for ranks 2, 4, and 8. For GO mf and GO bp, rank-16 runs confirm the convergence trend: rank-8 to rank-16 MAE is 3.7×10^{-3} (GO mf) and 5.2×10^{-3} (GO bp), consistent with the rank-4 to rank-8 reduction. The saturation pattern holds across all four ontologies.

D. Additional Theory and Diagnostics

D.1. General Boolean Queries

For any Boolean query ψ,

$$p_T(\psi) = \frac{\mathrm{WMC}(T \wedge \psi, p)}{\mathrm{WMC}(T, p)}.$$

D.2. Star Exactness

Theorem D.1 (Star exactness)**.** *For a parent π with independent children $c_1, \ldots, c_k$ under constraints $y_{c_j} \Rightarrow y_\pi$, the exact WMC marginal is*

$$\mu_\pi = \frac{p_\pi}{p_\pi + (1 - p_\pi) \prod_{j=1}^{k} (1 - p_{c_j})}.$$

Ontology	Terms	%mp	Max deg.	$\widehat{w}(G_T)$	Core	Modules	$\max_j \widehat{w}(A_j)$	Struct. cov.
GO cc	4,041	13.0	4	12	0.0%	1	12	100.0%
GO mf	10,131	20.0	7	37	5.0%	6,519	5	95.0%
GO bp	25,950	53.0	9	269	9.0%	7,249	12	91.0%
UBERON	14,977	24.0	7	46	5.0%	8,427	11	95.0%
ChEBI	205,592	24.7	14	738	2.0%	181,679	12	98.0%

Table 8: Unrestricted modular pre-screen. Structural coverage is $\sum_j |M_j|/|Y|$.

Proof. Condition on y_π. When $y_\pi = 1$, all child assignments are allowed, contributing $p_\pi \prod_j [p_{c_j} + (1 - p_{c_j})] = p_\pi$. When $y_\pi = 0$, every child must be false, contributing $(1 - p_\pi) \prod_j (1 - p_{c_j})$. Taking the ratio gives the stated formula. □

D.3. Diamond Calculation

For $T_\diamond$ with all four Bernoulli parameters equal to $1/2$, six assignments satisfy the implications; five have $y_a = 1$, giving $p_{T_\diamond}(y_a = 1) = 5/6$. The path-wise calculation gives $\widehat{\mu}_a = 9/10$ (Section 4).

D.4. Dense-Mode Cost

Exact dense-mode elimination costs

$$O\left(\sum_{j=1}^{s}\left[n_j 2^{\widehat{w}_j^{\partial}+1} + 2^{|B_j|}\right] + (|C| + s)2^{\widehat{w}_C+1}\right).$$

E. Ontology Instantiation Details

GO experiments: release 2021-11-16. UBERON: `uberon-basic.owl` release 2026-06-19. ChEBI: OBO data-version 252, dated 2026-05-01. Source paths, URLs, and SHA256 checksums are recorded in the artifact manifest.

F. Structure Decomposition Table

G. Formalisation: Lean 4 Development

The Lean 4 development (`Wmc.lean`, Lean 4.31) is a single Lean-core file with no dependence on `mathlib`. It defines worlds as Boolean lists, the true-path constraint as a conjunction of edge clauses $(\neg y_c \vee y_\pi)$, and closes every claim by `decide` using kernel-checkable rational arithmetic. Verified lemmas include `diamond_exact_5_6` $(5/6)$, `diamond_softor_9_10` $(9/10)$, and `diamond_softor_ne_exact` (inequality). The parametric star theorem is proved in a separate `mathlib` development with axiom footprint {`propext`, `Classical.choice`, `Quot.sound`}.